UNIVERSITATEA „POLITEHNICA” din BUCUREȘTI

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect

Programarea Interfețelor pentru Baze de Date

(1)

Student: Vincene Eduard

Grupa: 433A

Prof. coordonator:

Ș.l. Dr. Ing. Pupezescu Valentin

Cuprins

1. Cerințe 3
2. Tehnologii utilizate 3
   2.1 MySQL.....3
   2.2 Java Server Pages (JSP) 4
   2.3. Apache Tomcat 4
   2.4. HTML 4
3. Descrierea aplicației 5
   3.1 Baza de date 5
   3.2 Diagrama logică a bazei de date (Diagrama ERD) 6
4. Implementarea clasei JavaBean 7
   4.1 Inițializarea și conectarea la baza de date 7
   4.2 Închiderea conexiunii 8
   4.3 Manipularea Datelor 8
5. Implementarea JSP-urilor 11
6. Bibliografie.....18

1. Cerințe

Să se creeze două aplicații care să conțină o bază de date creată în sistemul de gestiune a bazelor de date MySQL și două interfețe la aceasta (baza de date este comună).

La crearea interfețelor se vor folosi două tehnologii (ex: JSP, Hibernate, JPA, .NET, Python etc.). Baza de date va fi compusă din tabelele: Filme și Categorii, cu asociere de tip M:N.

Interfețele realizate trebuie să permită utilizatorului să execute următoarele operații pe toate tabelele: vizualizare, adăugare, modificare și ștergere de date. Vizualizarea tabelelor de legătură va presupune vizualizarea datelor referite din celalte tabele.

Pentru tema primită, am ales 2 tehnologii diferite JSP și Python. În acest document, se va face referire la tehnologia JSP.

2. Tehnologii utilizate

2.1 MySQL

MySQL este un sistem de gestionare a bazelor de date relaționale open source care este utilizat în principal pentru aplicațiile online. MySQL poate crea și gestiona baze de date foarte utile (cum ar fi informații despre filme, inventar și multe altele), la fel ca alte sisteme, cum ar fi popularul Microsoft Access. În timp ce Microsoft Access, MySQL și alte sisteme de gestionare a bazelor de date servesc scopuri similare (de a găzdui datele), utilizarea diferă foarte mult. [1]

2.2 Java Server Pages (JSP)

JSP este o tehnologie esențială pentru dezvoltarea aplicațiilor web, ce combină puterea și extensibilitatea limbajului Java cu simplitatea și ușurința de folosire a scripturilor pe bază de etichete. O pagină JSP este un document text ce conține 2 tipuri de text: static, ce poate fi exprimat în orice tip de format bazat pe text (HTML, WML, XML, etc.), și conținutul JSP propriu-zis altfel spus dinamic.[2]

2.3. Apache Tomcat

Apache Tomcat este un server web open-source și un container de servlet-uri Java dezvoltat de Apache Software Foundation. Acesta furnizează un mediu de rulare pentru aplicații web Java, permițând implementarea și gestionarea lor într-un mod eficient și scalabil. Tomcat acționează ca un server web sau un mediu de execuție pentru aplicațiile Java, gestionând cererile HTTP de la clienți și facilitând executarea servlet-urilor și JSP-urilor pentru generarea răspunsurilor dinamice. [3]

2.4. HTML

Unul din primele elemente fundamentale ale WWW (World Wide Web) este HTML (Hypertext Markup Language), care descrie formatul primar în care documentele sunt distribuite și văzute pe Web. Multe din trăsăturile lui, cum ar fi independență față de platforma, structurarea formatării și legaturile hypertext, fac din el un foarte bun format pentru documentele Internet și Web. [4]

3. Descrierea aplicatiei

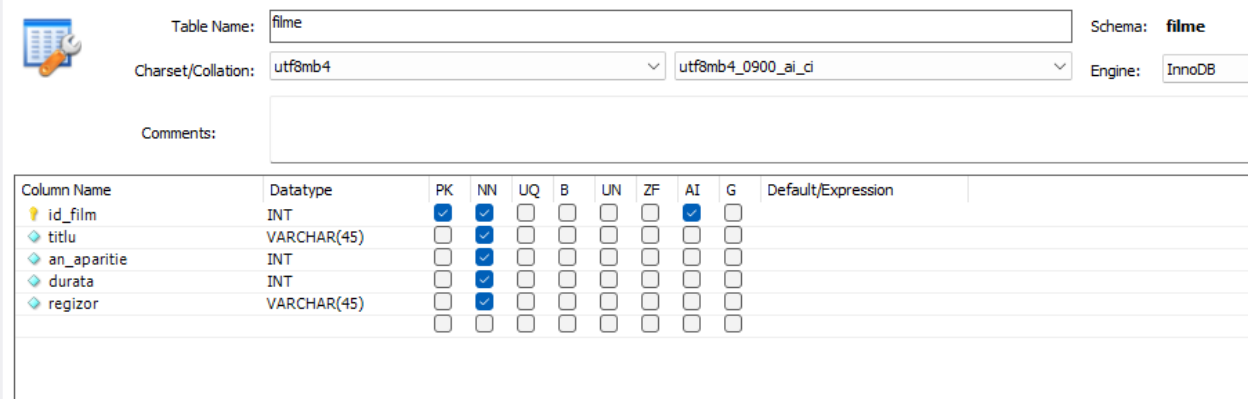
3.1 Baza de date

Tema individuală se bazează pe crearea unei baze de date ce are 2 tabele în asociere M:N. Tabelele sunt: filme si categorii.

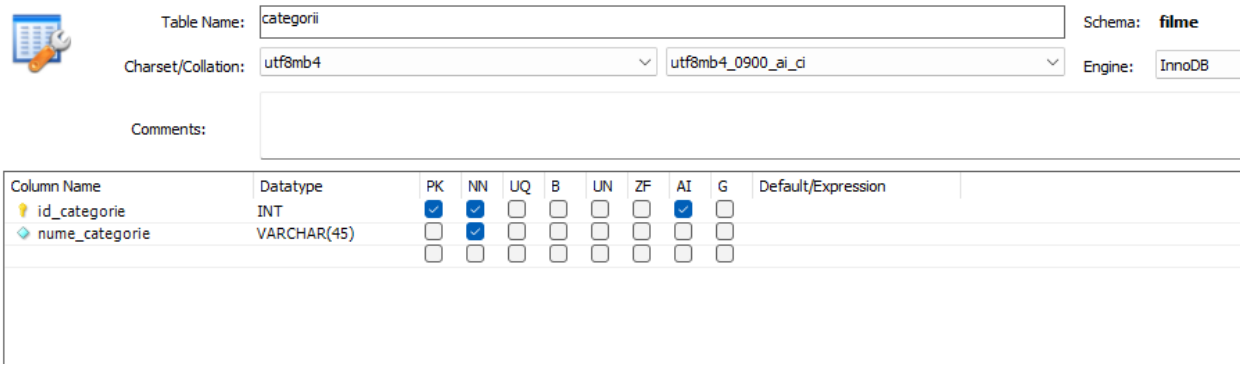
Asocierea M:N (Many-to-Many) are ca și caracteristică faptul că fiecărui element înregistrat într-o tabelă îi pot fi asociate mai multe elemente din cealaltă tabelă și invers. De exemplu, în acest caz, un film poate fi asociat mai multor categorii , așa cum și unei categorii îi pot fi asociate mai multe filme.

Pentru cele 2 tabele am ales cateva atribute caracteristice:

Pentru tabela filme: am ales ca și cheie primară id_film. Celelalte atribute sunt: titlu, an_aparitie, durata, regizor

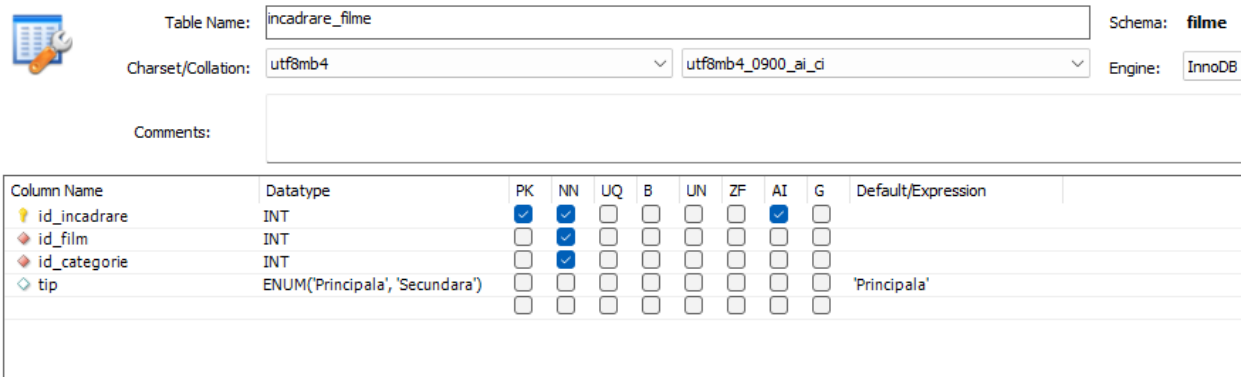


Pentru tabela categorii: am ales ca si cheie primara id_categorie. Celalalt atribut este nume_categorie



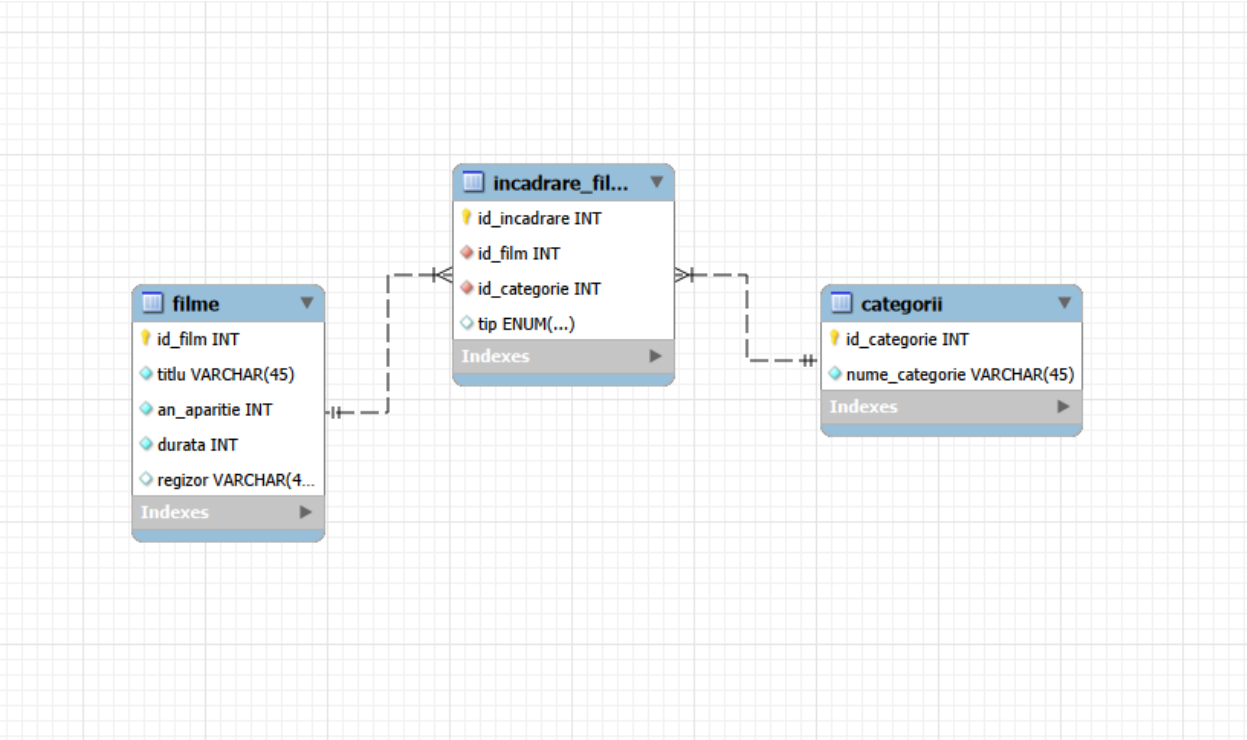
Pentru a crea o relație M:N, trebuie să se creeze o a treia tabelă denumită tabelă de joctiune, care împarte relația mai-mulți-la-mai-mulți în două relații unu-la-mai-mulți. Am ales ca și tabela de legatură tabela incadrare_filme. În această nouă tabelă, atributele ce au fost selectate ca și chei primare pentru tabelele anterioare vor deveni chei străine (FK) pentru tabela de legatură incadrare_filme.

Pentru această tabelă, am ales atributul id_incadrare ca și cheie primară. Restul atributelor sunt cheile străine id_film și id_categorie și atributul ce determină tip-ul



3.2 Diagrama logică a bazei de date (Diagrama ERD)

Diagrama ERD este o hartă vizuală a bazei de date. Aceasta ne arată tabelele proiectului și liniile care le unesc, ajutându-ne să înțelegem rapid structura logică și modul în care datele comunică între ele



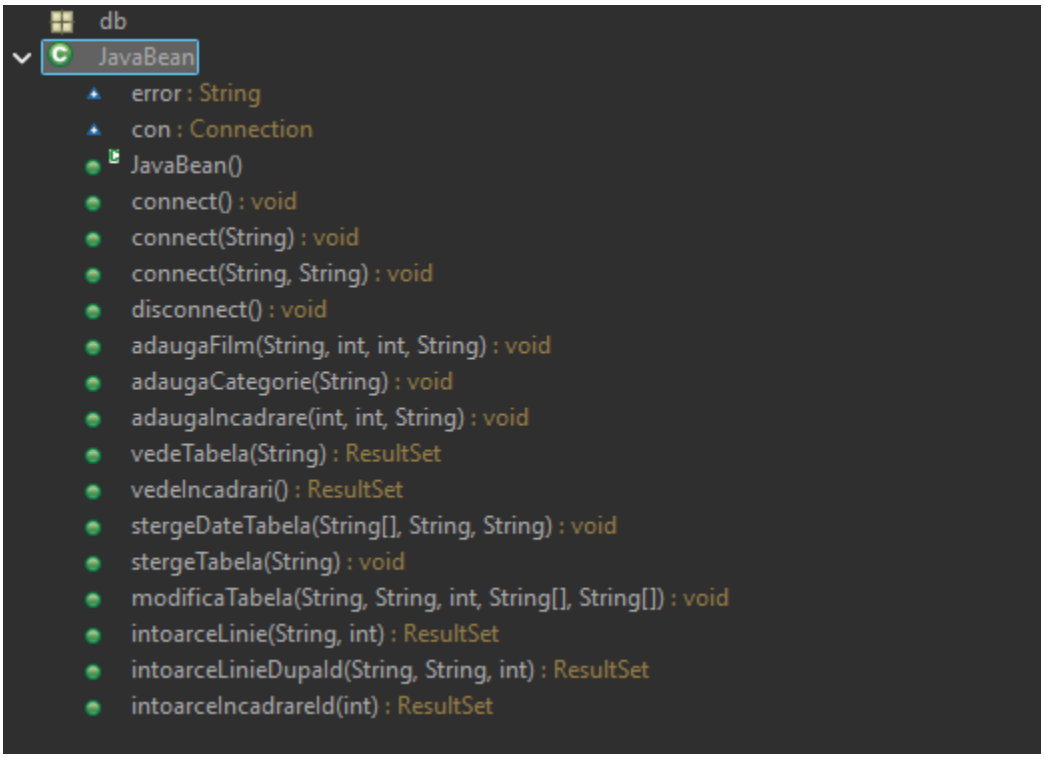
Asocierile dintre tabele:

- Între filme și categorii este o asociere M:N
- Între filme și incadrare_filme este o asociere 1:N
- Între categorii și incadrare_filme este o asociere 1:N

4. Implementarea clasei JavaBean

Clasa JavaBean servește ca un strat de intermediere între aplicația Java și baza de date MySQL, oferind metode pentru conectarea la baza de date, manipularea datelor și gestionarea excepțiilor

Diagrama UML a clasei JavaBean:



4.1 Inițializarea și conectarea la baza de date

Metodele connect(), connect(String bd) și connect(String bd, String ip) sunt utilizate pentru a stabili conexiunea cu baza de date MySQL, folosind driver-ul JDBC și credențialele specificate.

```
public void connect() throws ClassNotFoundException, SQLException, Exception {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/filme?useSSL=false", "root", "mere22");
    } catch (ClassNotFoundException cnfe) {
        error = "ClassNotFoundException: Nu s-a gasit driverul bazei de date.";
        throw new ClassNotFoundException(error);
    } catch (SQLException cnfe) {
        error = "SQLException: Nu se poate conecta la baza de date.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "Exception: A aparut o exceptie neprevazuta in timp ce se stabilea legatura la baza de date.";
        throw new Exception(error);
    }
}
```

4.2 Închiderea conexiunii

Metoda disconnect() închide conexiunea cu baza de date, asigurând gestionarea corespunzătoare a resurselor.

```
public void disconnect() throws SQLException {
    try {
        if (con != null) {
            con.close();
        }
    } catch (SQLException sqle) {
        error = ("SQLException: Nu se poate inchide conexiunea la baza de date.");
        throw new SQLException(error);
    }
}
```

4.3 Manipularea Datelor

Metodele adaugaFilm(), adaugaCategorie() și adaugaIncadrare () sunt folosite pentru a insera noi înregistrări în tabelele corespunzătoare.

```
public void adaugaFilm(String titlu, int an_aparitie, int durata, String regizor) throws SQLException, Exception {
    if (con != null) {
        try {
            Statement stmt;
            stmt = con.createStatement();
            stmt.executeUpdate("insert into filme(titlu, an_aparitie, durata, regizor) values('" + titlu + "', " + an_aparitie + ", " + durata + ", '" + regizor + "');");
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

Funcția vedeTabela(String tabel) execută o interogare de tip SELECT * pe tabela al cărei nume este transmis ca parametru, returnând toate înregistrările.

```
public ResultSet vedeTabela(String tabel) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = ("select * from `filme`.`" + tabel + "`;");
        Statement stmt = con.createStatement(/*ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY*/);
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

Pentru tabela de legătură „Incadrare”, se realizează un INTER JOIN între tabele.

```
public ResultSet vedeIncadrari() throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = ("SELECT i.id_incadrare, f.titlu, f.an_aparitie, f.durata, c.nume_categorie, i.tip, i.id_film, i.id_categorie " +
                "FROM filme f, categorii c, incadrare_filme i " +
                "WHERE f.id_film = i.id_film AND c.id_categorie = i.id_categorie;");
        Statement stmt = con.createStatement(/*ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY*/);
        rs = stmt.executeQuery(queryString);
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
}
```

Funcția stergeDateTabela() este utilizată pentru a șterge înregistrări. Primește un vector de ID-uri (de tip String), numele coloanei de identificare (Primary Key) și numele tabelei din care se face ștergerea.

```
public void stergeDateTabela(String[] primaryKeys, String tabela, String dupaID) throws SQLException, Exception {
    if (con != null) {
        try {

            long aux;
            PreparedStatement delete;
            delete = con.prepareStatement("DELETE FROM " + tabela + " WHERE " + dupaID + "=?;");
            for (int i = 0; i < primaryKeys.length; i++) {
                aux = java.lang.Long.parseLong(primaryKeys[i]);
                delete.setLong(1, aux);
                delete.execute();
            }
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        } catch (Exception e) {
            error = "A aparut o exceptie in timp ce erau sterse inregistrarile.";
            throw new Exception(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

Funcția modificaTabela() este responsabilă de executarea operațiunilor de actualizare (UPDATE) asupra înregistrărilor dintr-o tabelă dată. Pentru recuperarea datelor, metodele intoarceLinie() și intoarceLinieDupaId() permit selecția și returnarea unor rânduri specifice, identificate prin index sau cheie primară

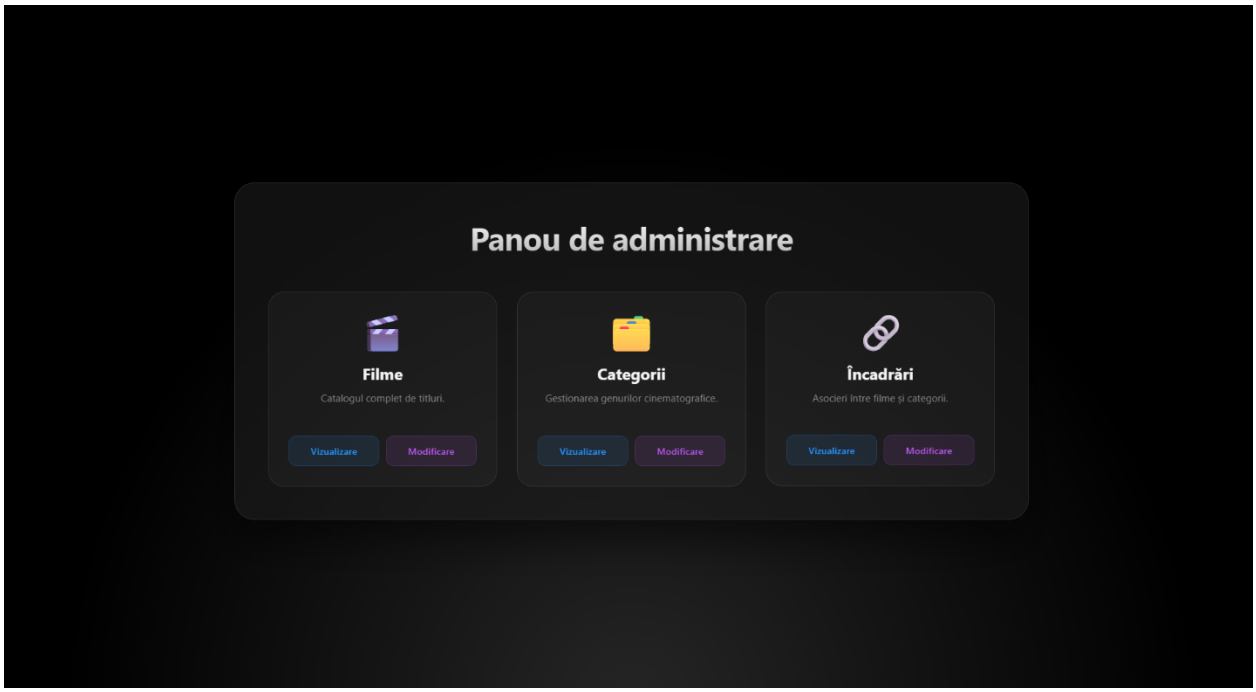
```
public ResultSet intoarceLinie(String tabela, int ID) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        String queryString = ("SELECT * FROM " + tabela + " where id_film=" + ID + ";");
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
        rs = stmt.executeQuery(queryString); //sql exception
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
} // end of intoarceLinie()
```

```
public void modificaTabela(String tabela, String IDTabela, int ID, String[] campuri, String[] valori) throws SQLException, Exception {
    String update = "update " + tabela + " set ";
    String temp = "";
    if (con != null) {
        try {
            for (int i = 0; i < campuri.length; i++) {
                if (i != (campuri.length - 1)) {
                    temp = temp + campuri[i] + "='" + valori[i] + "', ";
                } else {
                    temp = temp + campuri[i] + "='" + valori[i] + "' where " + IDTabela + " = '" + ID + "';";
                }
            }
            update = update + temp;
            Statement stmt;
            stmt = con.createStatement();
            stmt.executeUpdate(update);
        } catch (SQLException sqle) {
            error = "ExceptieSQL: Reactualizare nereusita; este posibil sa existe duplicate.";
            throw new SQLException(error);
        }
    } else {
        error = "Exceptie: Conexiunea cu baza de date a fost pierduta.";
        throw new Exception(error);
    }
}
```

```
public ResultSet intoarceLinieDupaId(String tabela, String denumireId, int ID) throws SQLException, Exception {
    ResultSet rs = null;
    try {
        // Executa interogarea
        String queryString = ("SELECT * FROM " + tabela + " where " + denumireId + "=" + ID + ";");
        Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
        rs = stmt.executeQuery(queryString); //sql exception
    } catch (SQLException sqle) {
        error = "SQLException: Interogarea nu a fost posibila.";
        throw new SQLException(error);
    } catch (Exception e) {
        error = "A aparut o exceptie in timp ce se extrageau datele.";
        throw new Exception(error);
    }
    return rs;
} // end of intoarceLinieDupaId()
```

5. Implementarea JSP-urilor

Pagina principală a interfeței (index.html), realizată în HTML și CSS (style.css), servește drept panou de administrare. Aceasta oferă acces rapid la cele trei secțiuni principale ale bazei de date: **Filme**, **Categorii** și **Încadrări**.



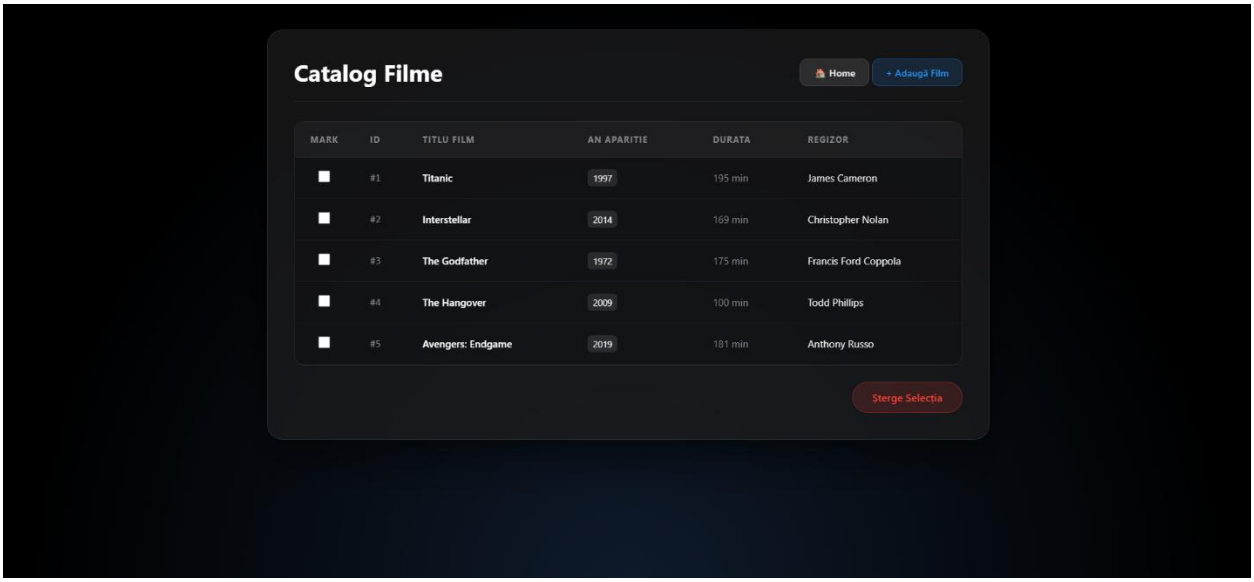
Pentru fiecare tabelă există două opțiuni distincte care separă fluxul de lucru:

1. Butonul „Vizualizare”: Permite vizualizarea completă a catalogului, ștergerea înregistrărilor și adăugarea de noi date.

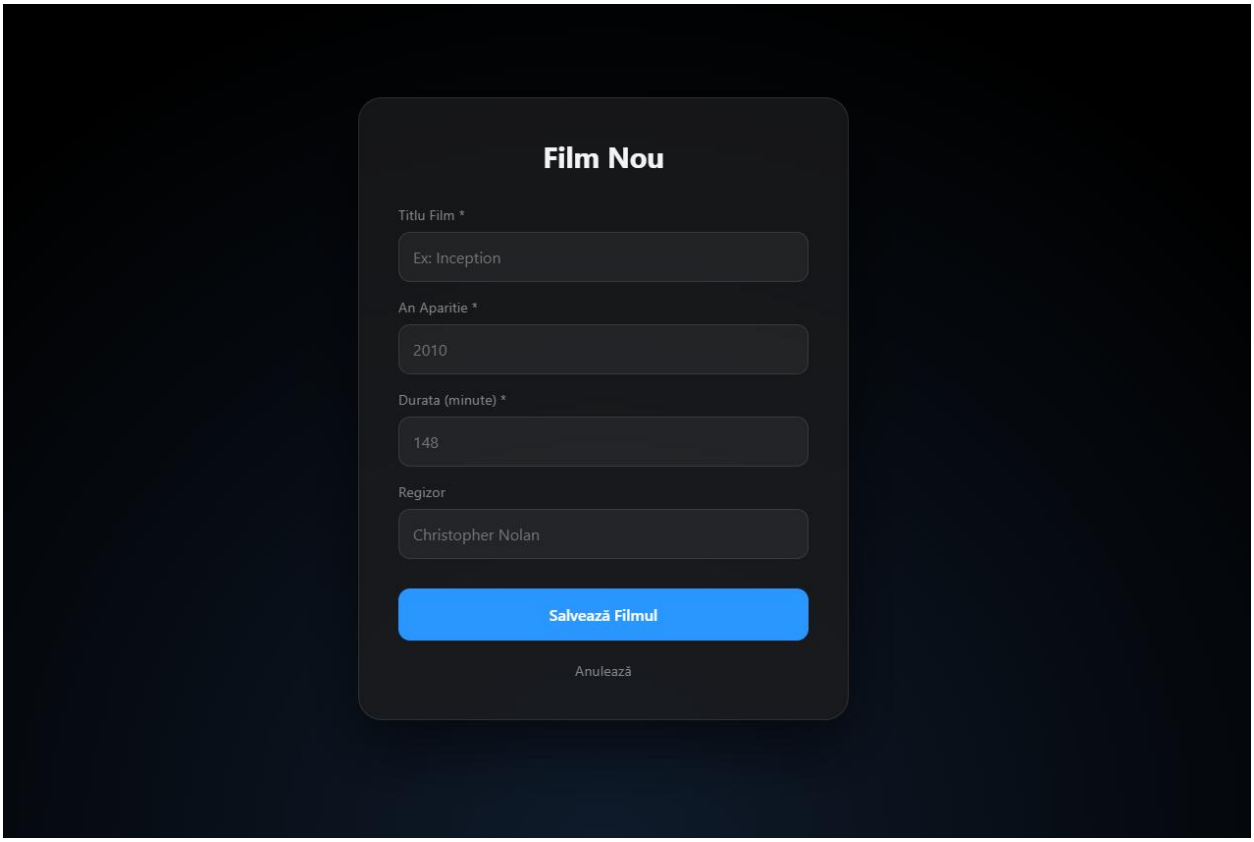
2. Butonul „Modificare”: Permite selectarea unei înregistrări specifice pentru a-i edita datele.

Accesând butonul „Vizualizare” (ex: pentru Filme), utilizatorul este direcționat către fișierul **tabela_Filme.jsp**. Aici datele sunt extrase din baza de date și afișate într-un tabel stilizat.

```
<table>
    <thead>
        <tr>
            <th style="width: 50px; text-align: center;">Mark</th>
            <th style="width: 80px;">ID</th>
            <th>Titlu Film</th>
            <th>An Aparitie</th>
            <th>Durata</th>
            <th>Regizor</th>
        </tr>
    </thead>
    <tbody>
        <%
        jb.connect();
        ResultSet rs = jb.vedeTabela("filme");
        long x;
        while (rs.next()) {
            x = rs.getInt("id_film");
        %>
```



Dacă se dorește adăugarea unui nou film, utilizatorul apasă butonul **„+ Adaugă Film”**. Acest fragment de cod JSP se ocupă cu preluarea parametrilor dintr-o cerere HTTP și utlizarea acestor parametri pentru a adăuga un nou film intr-o bază de date.



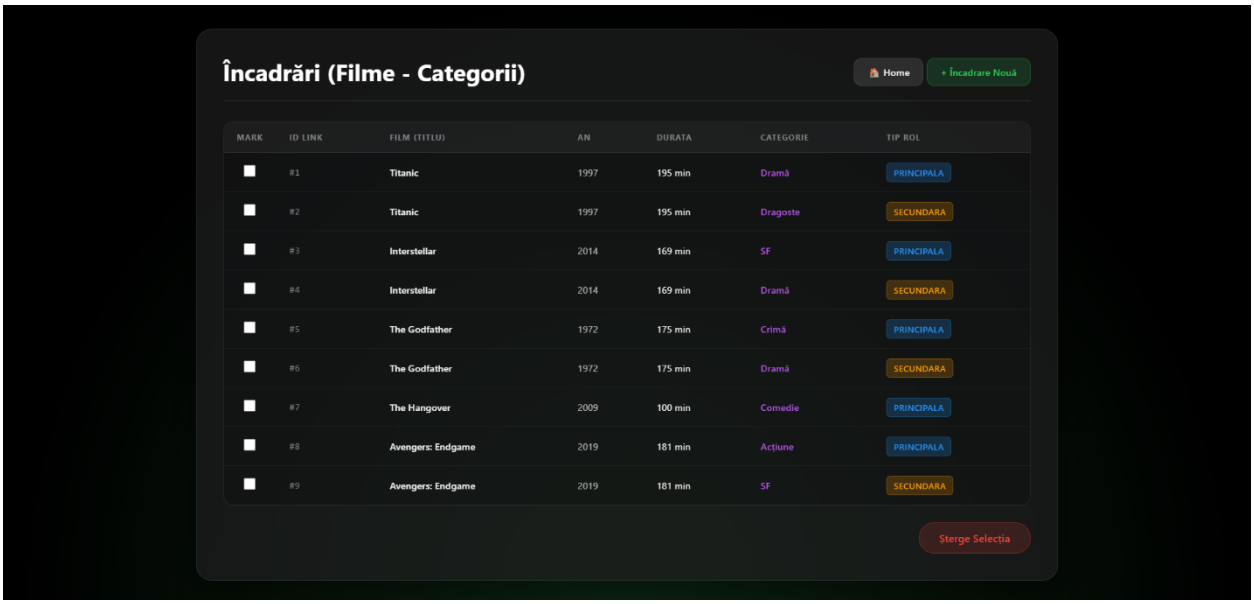

```
<%
    String titlu = request.getParameter("titlu");
    String an_aparitie = request.getParameter("an_aparitie");
    String durata = request.getParameter("durata");
    String regizor = request.getParameter("regizor");

    if (titlu != null && an_aparitie != null && durata != null && !an_aparitie.isEmpty() && !durata.isEmpty()) {
        jb.connect();
        int an = Integer.parseInt(an_aparitie);
        int dur = Integer.parseInt(durata);
        jb.adaugaFilm(titlu, an, dur, regizor);
        jb.disconnect();
%>
```

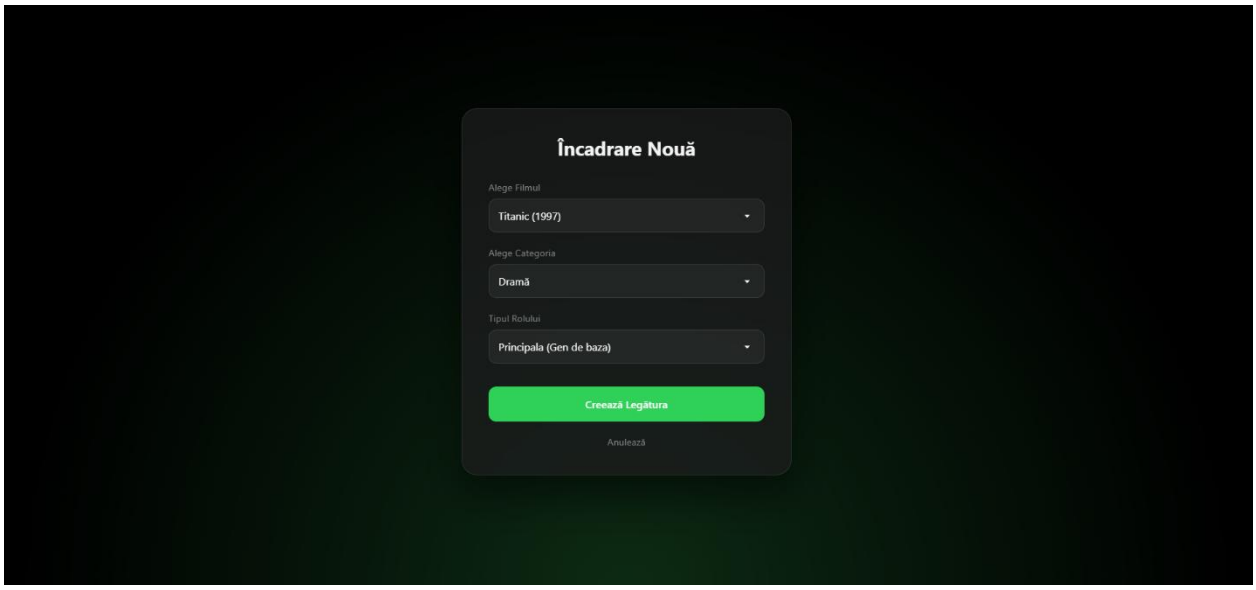
Codul din spatele fișierului nou_Filme.jsp preia parametrii din formular (request.getParameter) și apelează metoda adaugaFilm din bean pentru a face insert în baza de date.

Similar se procedeaza si pentru tabela Categorii

Adăugarea în tabela incadrare_filme este similară cu celelalte operațiuni, dar diferă prin modul de introducere a datelor. Deoarece este o tabelă de legătură, aplicația impune selectarea **filmului** și a **categoriei** dorite. De asemenea, utilizatorul trebuie să specifice **tipul asocierii** alegând una dintre valorile predefinite: „Principal” sau „Secundar”

Tabela Incadrari:





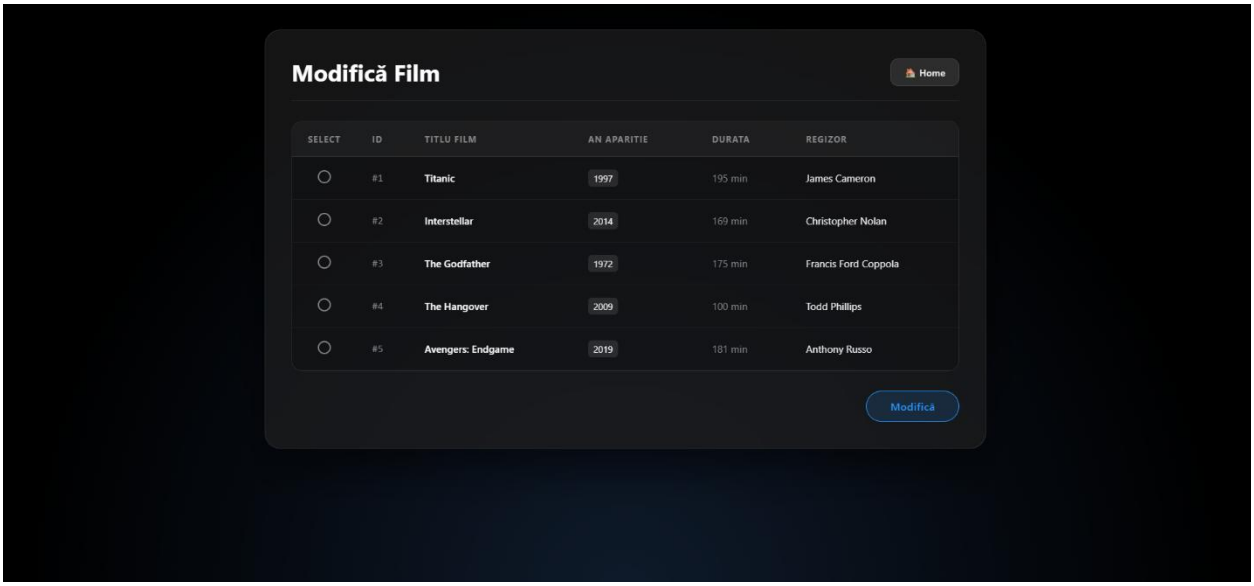
Pentru a șterge înregistrări, utilizatorul bifează căsuțele și apasă butonul „Șterge Selecția”. Formularul trimite datele către fișierul de procesare **sterge_Filme.jsp**.

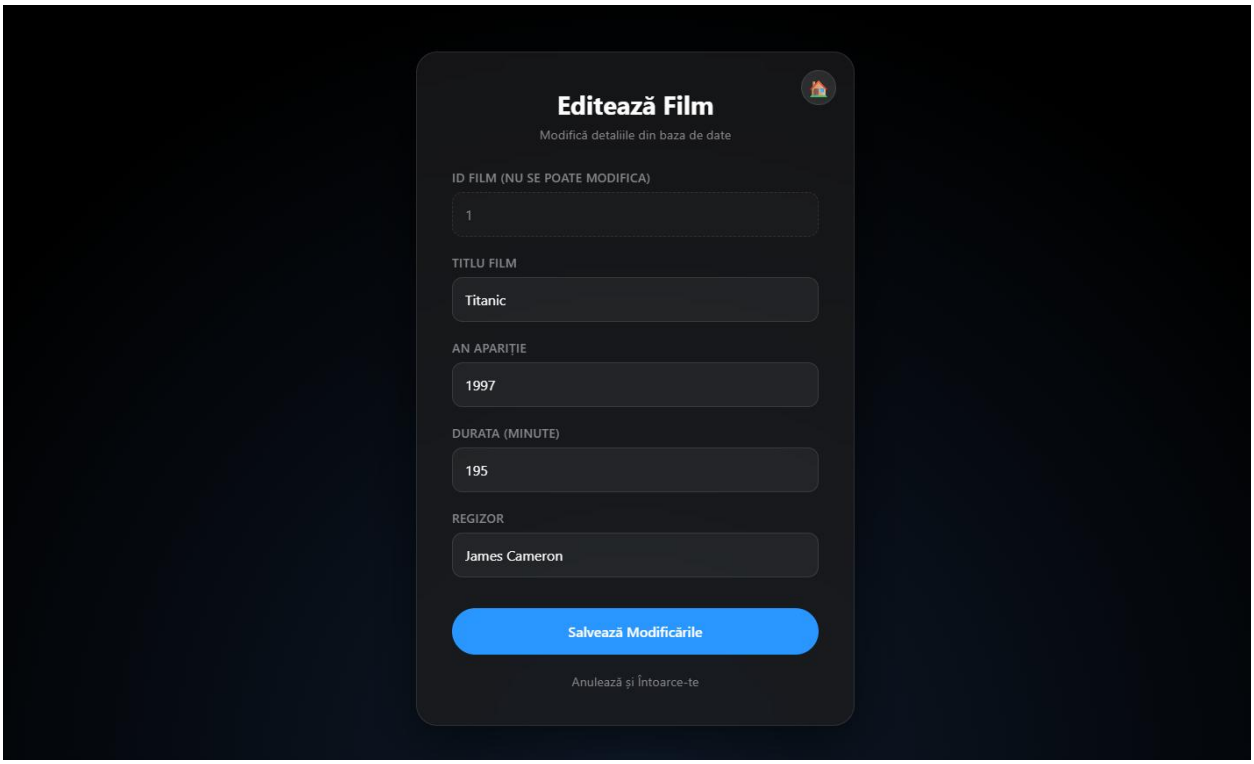
```
<%
    String[] s = request.getParameterValues("primarykey");

    if (s != null && s.length > 0) {
        jb.connect();
        jb.stergeDateTabela(s, "filme", "id_film");
        jb.disconnect();
%>
```

Pentru a modifica datele, utilizatorul trebuie sa revina la pagina de start.

Apoi, accesând butonul „Modificare” (ex: pentru Filme), utilizatorul este direcționat către fișierul **modifica_Filme.jsp**. Acest fragment de cod JSP se ocupă cu preluarea parametrilor dintr-o cerere HTTP și utilizarea acestor parametri pentru a modifica datele alese de utilizator din bază de date.

```
<%
    jb.connect();

    // 1. Preluam datele trimise de m1_Filme.jsp
    int aux = java.lang.Integer.parseInt(request.getParameter("id_film"));
    String titlu = request.getParameter("titlu");
    String an_aparitie = request.getParameter("an_aparitie");
    String durata = request.getParameter("durata");
    String regizor = request.getParameter("regizor");

    // 2. Pregatim vectorii pentru functia generica
    String[] valori = {titlu, an_aparitie, durata, regizor};
    String[] campuri = {"titlu", "an_aparitie", "durata", "regizor"};

    // 3. Executam UPDATE-ul
    jb.modificaTabela("filme", "id_film", aux, campuri, valori);

    jb.disconnect();
%>
```


Modifică Film
Home
SELECT
ID
TITLU FILM
AN APARITIE
DURATA
REGIZOR
#1 Titanic 1997 195 min James Cameron
#2 Interstellar 2014 169 min Christopher Nolan
#3 The Godfather 1972 175 min Francis Ford Coppola
#4 The Hangover 2009 100 min Todd Phillips
#5 Avengers: Endgame 2019 181 min Anthony Russo
Modifică


Editează Film
Modifică detaliile din baza de date
ID FILM (NU SE POATE MODIFICA)
1
TITLU FILM
Titanic
AN APARIȚIE
1997
DURATA (MINUTE)
195
REGIZOR
James Cameron
Salvează Modificările
Anulează și Întoarce-te

6. Bibliografie

[1] - https://www.nav.ro/blog/ce-este-mysql/

[2] - Tehnologii de programare în Internet (http://discipline.elcom.pub.ro/tpi)

[3] - https://tomcat.apache.org/

[4] - https://web.ceiti.md/lesson.php?id=1